

Project Report: Improving a Vector-Space IR System (Parts 1–4)

Name: _____

Roll No.: _____

Course: CS6370 – Natural Language Processing
Dataset: Cranfield (1400 documents, 225 queries)

Abstract

This document is

Contents

1	Introduction	3
1.1	Objectives	3
2	Part 1: Working out a Toy IR System (Ambiguity Focus)	3
2.1	Preprocessing and Inverted Index	3
2.1.1	Tokenization and Stopword Removal	3
2.1.2	Inverted Index Construction	3
2.2	TF-IDF Matrix	4
2.3	Boolean Retrieval	5
2.4	Cosine Similarity and Ranking	5
2.4.1	Query Representation using TF-IDF	5
2.4.2	Cosine Similarity Computation	5
2.4.3	Ranking of Documents	6
2.4.4	Justification	6
2.5	Analysis of Word Sense Ambiguity	6
2.5.1	Ideal Retrieval	6
2.5.2	System Retrieval	6
2.5.3	Explanation of Word Sense Ambiguity	6
3	Part 2: Building and Extending an IR System [Implementation]	7
3.1	Retrieval using TF-IDF Vector Space Model	7
3.1.1	Preprocessing	7
3.1.2	Index Construction	7
3.1.3	TF-IDF Representation	7
3.1.4	Query Representation	7
3.1.5	Document Ranking	8
4	Part 3: Evaluating your IR System	8

5	Part 4: Analysis	9
5.1	Limitations of vector space models	9
5.2	Concrete examples from your retrieval results	9
5.2.1	Example queries	9
5.2.2	Queries from Dataset	10
5.3	The Zero-Result Problem (OOV Issue):	11
5.3.1	Out-of-Vocabulary term in query	11
5.3.2	Effect on Retrieval Results	11

1 Introduction

This project implements and analyses an Information Retrieval (IR) system based on the Vector Space Model (VSM) using the Cranfield dataset. Parts 1–4 of the project cover: preprocessing and indexing, the TF–IDF representation and retrieval, evaluation of the basic IR system using standard metrics, and an analysis of VSM limitations with concrete examples. Subsequent parts will investigate improvements to the baseline system.

1.1 Objectives

- Build a reproducible pipeline for preprocessing (segmentation, tokenization, stopword removal, stemming/lemmatization) and inverted index construction.
- Represent documents and queries with TF–IDF vectors and implement retrieval using cosine similarity and ranking.
- Evaluate retrieval with Precision@k, Recall@k, F0.5@k, AP@k, nDCG@k, MAP and MRR.
- Analyse limitations of VSM and propose concrete improvements for the project.

2 Part 1: Working out a Toy IR System (Ambiguity Focus)

Given sample data for the part

- d1: The star in our solar system provides heat and light.
- d2: That Hollywood star walked the red carpet for the movie premiere.
- d3: Astronomers observe distant stars and galaxies using telescopes.

2.1 Preprocessing and Inverted Index

2.1.1 Tokenization and Stopword Removal

After performing tokenization, lemmatization, and stopword removal on the given documents, the processed tokens are:

- d_1 : {star, solar, system, provide, heat, light}
- d_2 : {hollywood, star, walk, red, carpet, movie, premiere}
- d_3 : {astronomer, observe, distant, star, galaxy, use, telescope}

2.1.2 Inverted Index Construction

The inverted index maps each term to the set of documents in which it appears.

Term	Document IDs
star	$\{d_1, d_2, d_3\}$
solar	$\{d_1\}$
system	$\{d_1\}$
provide	$\{d_1\}$
heat	$\{d_1\}$
light	$\{d_1\}$
hollywood	$\{d_2\}$
walk	$\{d_2\}$
red	$\{d_2\}$
carpet	$\{d_2\}$
movie	$\{d_2\}$
premiere	$\{d_2\}$
astronomer	$\{d_3\}$
observe	$\{d_3\}$
distant	$\{d_3\}$
galaxy	$\{d_3\}$
use	$\{d_3\}$
telescope	$\{d_3\}$

2.2 TF-IDF Matrix

The TF-IDF representation is constructed using:

$$w_{i,j} = tf_{i,j} \times IDF_i$$

where,

$$IDF_i = \log\left(\frac{D}{df_i}\right), \quad D = 3$$

Term	Term Frequencies (TF)				IDF Calculations			Final Weights ($w_{i,j}$)			
	Q	d_1	d_2	d_3	df_i	D/df_i	IDF_i	Q	d_1	d_2	d_3
star	1	1	1	1	3	1	0	0	0	0	0
solar	0	1	0	0	1	3	$\log(3)$	0	$\log(3)$	0	0
system	0	1	0	0	1	3	$\log(3)$	0	$\log(3)$	0	0
provide	0	1	0	0	1	3	$\log(3)$	0	$\log(3)$	0	0
heat	0	1	0	0	1	3	$\log(3)$	0	$\log(3)$	0	0
light	1	1	0	0	1	3	$\log(3)$	$\log(3)$	$\log(3)$	0	0
hollywood	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
walk	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
red	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
carpet	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
movie	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
premiere	0	0	1	0	1	3	$\log(3)$	0	0	$\log(3)$	0
astronomer	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$
observe	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$
distant	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$
galaxy	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$
use	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$
telescope	0	0	0	1	1	3	$\log(3)$	0	0	0	$\log(3)$

Table 1: Complete Term Frequencies, Document Frequencies, and Final TF-IDF Weights

2.3 Boolean Retrieval

Consider the query: “*star light*”.

After tokenization and preprocessing, the query is represented as a list of terms:

$$Q = [\text{'star'}, \text{'light'}]$$

Using the inverted index, we retrieve the postings lists (document IDs) associated with each term:

- $\text{'star'} \rightarrow \{1, 2, 3\}$
- $\text{'light'} \rightarrow \{1\}$

To satisfy the query, a document needs to contain at least one of the terms, representing an implicit **OR** operation. Therefore, we compute the union of the retrieved postings lists to find all relevant documents:

$$\text{Result} = \{1, 2, 3\} \cup \{1\} = \{1, 2, 3\}$$

Final Answer: Documents d_1 , d_2 , and d_3 all match the query, as they each contain at least one of the terms *star* or *light*.

2.4 Cosine Similarity and Ranking

2.4.1 Query Representation using TF-IDF

The query is: “*star light*”.

After preprocessing, the query terms are: {star, light}.

Using TF-IDF weights:

- $IDF(\text{star}) = \log(3/3) = 0$
- $IDF(\text{light}) = \log(3/1) = \log(3)$

Thus, the query vector is:

$$Q = [0, \log(3)]$$

2.4.2 Cosine Similarity Computation

Cosine similarity is given by:

$$\text{cosine}(Q, d) = \frac{Q \cdot d}{\|Q\| \|d\|}$$

For d_1 :

$$Q \cdot d_1 = (\log(3)) \times (\log(3)) = (\log(3))^2$$

$$\|Q\| = \log(3), \quad \|d_1\| = \sqrt{6(\log(3))^2} = \sqrt{6} \log(3)$$

$$\text{cosine}(Q, d_1) = \frac{(\log(3))^2}{\log(3) \cdot \sqrt{6} \log(3)} = \frac{1}{\sqrt{6}} \approx 0.408$$

For d_2 :

$$Q \cdot d_2 = 0 \Rightarrow \text{cosine}(Q, d_2) = 0$$

For d_3 :

$$Q \cdot d_3 = 0 \Rightarrow \text{cosine}(Q, d_3) = 0$$

2.4.3 Ranking of Documents

Based on similarity scores:

$$d_1 > d_2 = d_3$$

2.4.4 Justification

Document d_1 has the highest similarity score because it contains the term *light*, which has a high IDF value. The other documents do not contain this term, resulting in zero similarity.

2.5 Analysis of Word Sense Ambiguity

Consider the query: "*movie star*".

2.5.1 Ideal Retrieval

Ideally, document d_2 should be retrieved, as it refers to a *movie star* in the context of a Hollywood actor.

2.5.2 System Retrieval

The system retrieves documents based on term matching. Since the word *star* appears in all documents, multiple documents may be retrieved instead of only d_2 .

2.5.3 Explanation of Word Sense Ambiguity

The term *star* has multiple meanings:

- Astronomical object (in d_1 and d_3)
- Movie celebrity (in d_2)

The retrieval system does not understand the context or meaning of words and treats all occurrences of *star* equally. As a result, it cannot distinguish between different senses of the word.

This leads to incorrect or irrelevant documents being retrieved. This problem is known as **word sense ambiguity**.

3 Part 2: Building and Extending an IR System [Implementation]

3.1 Retrieval using TF-IDF Vector Space Model

The retrieval component of the Information Retrieval (IR) system is implemented using the TF-IDF vector space model. The system processes documents and queries through a common pipeline to ensure consistent representation.

3.1.1 Preprocessing

Each document goes through following preprocessing steps:

- Sentence segmentation
- Tokenization
- Inflection reduction (lemmatization)
- Stopword removal

These steps convert raw text into a structured form consisting of meaningful tokens.

3.1.2 Index Construction

An inverted index is built to store term frequencies for each document. For every term, the index maintains a mapping of document IDs and their corresponding term frequencies.

The Inverse Document Frequency (IDF) for each term is computed as:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

where N is the total number of documents and $df(t)$ is the number of documents containing term t .

3.1.3 TF-IDF Representation

Each document is represented as a TF-IDF vector. The weight of a term in a document is given by:

$$w_{i,j} = TF_{i,j} \times IDF_j$$

A matrix is constructed where:

- Rows correspond to documents
- Columns correspond to terms in the vocabulary

The document vectors are normalized using L2 normalization to ensure fair comparison:

$$d_i = \frac{d_i}{\|d_i\|}$$

3.1.4 Query Representation

Queries are processed using the same preprocessing pipeline. A TF-IDF vector is constructed for each query using the same vocabulary and IDF values.

The query vector is also normalized:

$$q = \frac{q}{\|q\|}$$

3.1.5 Document Ranking

The relevance of documents with respect to a query is computed using cosine similarity:

$$\text{cosine}(q, d) = q \cdot d$$

The similarity scores are computed using vector dot product, and documents are ranked in descending order of these scores. This ranking represents the predicted relevance of documents to the query.

4 Part 3: Evaluating your IR System

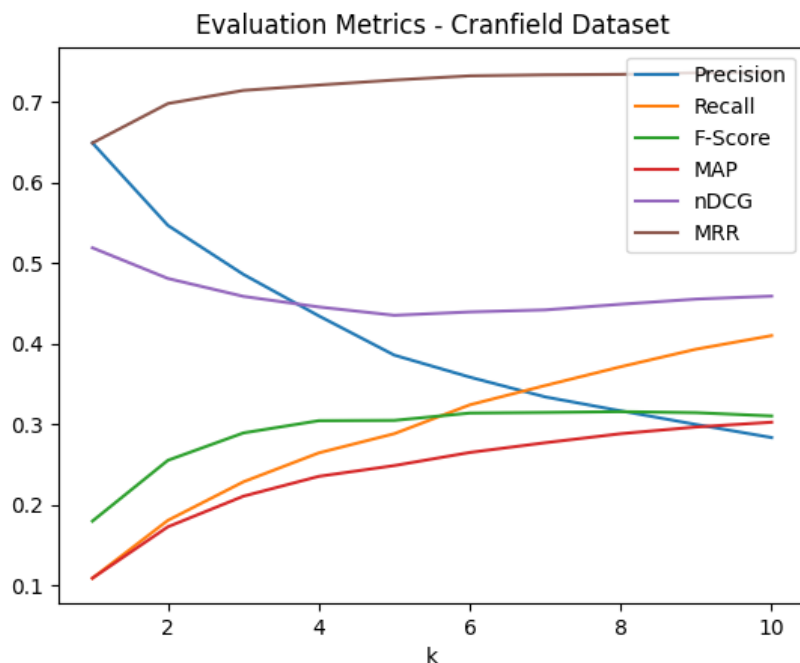


Figure 1: Evaluation Metrics - Cranfield Dataset

Observations on Evaluation Metrics

Based on the performance of the Information Retrieval (IR) system on the Cranfield Dataset (Figure 1), we can observe the following trends as the number of retrieved documents (k) increases from 1 to 10:

- **Precision-Recall Trade-off:** A distinct inverse relationship is present. As k increases, **Precision** drops sharply from approximately 0.65 to 0.28, while **Recall** climbs steadily from roughly 0.11 to 0.41.
- **F-Score Plateau:** The **F-Score** rises initially but plateaus around $k = 4$ at a value of roughly 0.30, indicating diminishing returns in the harmonic mean of Precision and Recall beyond the top 4 documents.
- **Mean Reciprocal Rank (MRR):** **MRR** remains exceptionally high, starting around 0.65 and plateauing above 0.70. This demonstrates that the system is highly proficient at retrieving the first relevant document at rank 1 or 2.

- **nDCG and MAP:** **nDCG** starts strong above 0.50, dips slightly around $k = 5$, and slowly recovers, showing solid overall ranking quality. Meanwhile, **MAP** increases gradually alongside Recall, reflecting a slow but steady accumulation of precision across multiple relevant documents.

5 Part 4: Analysis

5.1 Limitations of vector space models

Key Limitations of VSM include:

- **Lack of semantic understanding:** VSM treats each word as an independent and orthogonal dimension, ignoring semantic relationships such as synonymy and contextual similarity. As a result, it fails to capture true semantic meaning.
- **High dimensionality and computational cost:** The vocabulary size determines the dimensionality of the space, which can be extremely large. Storing and operating on sparse TF-IDF matrices becomes computationally expensive in both memory and processing time.
- **Poor scalability:** Adding new documents or terms often requires recomputing TF-IDF values across the entire corpus, making updates inefficient for large-scale or dynamic datasets.
- **Word sense ambiguity:** VSM cannot distinguish between different meanings of the same word (polysemy), leading to inaccurate similarity calculations.
- **Out-of-vocabulary (OOV) problem:** Words not present in the training corpus cannot be represented, limiting the model's ability to handle new or rare terms.
- **Lack of contextual representation:** Word order and context are ignored, meaning phrases with different meanings but similar word sets may be treated as equivalent.
- **Sparse representations:** Document vectors are typically sparse, which can reduce the effectiveness of similarity measures and increase storage overhead.

5.2 Concrete examples from your retrieval results

5.2.1 Example queries

- **Query:** *slip-flow heat transfer in internal channels*

Top five documents: 550, 941, 270, 1221, 102

Closest Cranfield query: 9 (*internal / slip flow / heat transfer studies*)

Analysis: Only document 550 is clearly relevant, showing a strong lexical match with the query. The remaining results are largely generic heat-transfer or flow-related papers. This highlights that VSM can capture surface-level term overlap but often fails to retrieve documents matching the precise subtopic, reflecting its limitation in semantic and phrase-level understanding.

- **Query:** *transition detection in hypersonic wakes behind slender bodies*

Top five documents: 1141, 536, 154, 976, 289

Closest Cranfield queries: 38, 40 (*hypersonic wake transition*)

Analysis: Documents 536 and 976 are relevant, but the top-ranked document (1141) is only broadly related. This shows that VSM may rank less precise results higher due to lexical overlap, lacking understanding of full physical context and relationships.

- Query:** *replace vibrational shapes with static deflection shapes for flutter prediction*
Top five documents: 52, 878, 593, 391, 15
Closest Cranfield query: 64 (*static vs vibrational shapes for flutter prediction*)
Analysis: Only document 391 is clearly relevant (rank 4). VSM captures shared terms like *flutter* and *shapes* but fails to model the comparison and substitution intent, highlighting weak relational semantics.
- Query:** *shock-induced boundary-layer separation*
Top five documents: 265, 311, 1187, 187, 996
Closest Cranfield query: 90 (*shock-induced boundary-layer separation*)
Analysis: Documents 311 and 1187 are relevant, with 1187 being highly specific. However, a non-relevant document is ranked first. This reflects VSM’s inability to treat multi-word technical phrases as cohesive concepts, leading to suboptimal ranking.
- Overall Observation:** VSM performs well with strong lexical overlap but struggles with semantic specificity, phrase-level meaning, and optimal ranking. It relies on bag-of-words matching and does not effectively capture intent or relationships between terms.

5.2.2 Queries from Dataset

- Query 9:** *papers on internal / slip flow / heat transfer studies*
Top five documents: 102, 45, 846, 763, 398
Relevant in top 5: 0/5 (*True relevant:* 534, 550, 21, 22)
Analysis: Complete failure case. The model retrieves generic heat-transfer papers and misses the specific slip-flow concept, highlighting its inability to capture phrase-level meaning.
- Query 39:** *how can one detect transition phenomena in boundary layers*
Top five documents: 37, 272, 1205, 295, 1264
Relevant in top 5: 1/5 (Doc 272 at rank 2)
Analysis: The model identifies the general topic but fails to strongly rank the most relevant boundary-layer transition work, indicating weak discrimination among similar documents.
- Query 40:** *how can one detect transition phenomena in hypersonic wakes*
Top five documents: 536, 37, 1141, 272, 295
Relevant in top 5: 2/5 (Docs 536, 272)
Analysis: Inclusion of *hypersonic* improves retrieval specificity. However, irrelevant results (e.g., 1141) still appear, showing limited contextual understanding.
- Query 51:** *boundary layers on very slender bodies of revolution in continuum flow*
Top five documents: 494, 326, 1301, 94, 23
Relevant in top 5: 4/5
Analysis: Strongest result. High lexical overlap enables accurate retrieval, though minor confusion (e.g., 1301) reflects limited fine-grained distinction.
- Query 64:** *static deflection shapes instead of vibrational shapes for flutter prediction*
Top five documents: 52, 390, 878, 593, 834
Relevant in top 5: 1/5 (Doc 390 at rank 2)
Analysis: VSM captures shared terms (*flutter*, *shapes*) but fails to model comparison and substitution, demonstrating weak relational semantics.

- **Query 90:** *recent data on shock-induced boundary-layer separation*
Top five documents: 265, 311, 1187, 187, 996
Relevant in top 5: 2/5 (Docs 311, 1187)
Analysis: Correct topic is retrieved, but optimal ranking is not achieved. Indicates limitations in prioritizing the most specific document within closely related results.
- **Overall Observation:** VSM performs well with strong lexical overlap (e.g., Query 51) but struggles with semantic precision, multi-word concepts, and intent modeling. It relies heavily on surface-level term matching, leading to both retrieval and ranking errors.

5.3 The Zero-Result Problem (OOV Issue):

5.3.1 Out-of-Vocabulary term in query

If an out-of-vocabulary term:

- The query is effectively reduced to only the words that are already in the document vocabulary. If the query still has some in-vocabulary terms, retrieval is based only on those remaining terms.
- If all query terms are OOV, the query vector becomes all zeros. With a zero query vector, cosine similarity with every document is zero, so the ranking is not meaningful and is essentially a tie-broken output.
- This is a limitation because the system cannot recover the intended meaning of the query unless it supports synonym expansion, spelling correction, or approximate matching.

5.3.2 Effect on Retrieval Results

In VSM, out-of-vocabulary (OOV) terms are ignored during scoring. The query is reduced to only in-vocabulary terms; if all terms are OOV, document similarities become zero, leading to arbitrary rankings that appear valid but lack meaningful grounding.

Example: *what corrections are needed for a liftbody in a propwash flowfield inside a test duct*
Closest Cranfield query: 81 (*wind-tunnel interference*)

Relevant documents: 631, 672, 799

Top five documents: 669, 450, 1270, 607, 653

Precision@5: 0

Analysis: Only document 669 is loosely related; none of the truly relevant documents are retrieved. The failure is lexical: key terms like *propwash* are absent from the vocabulary, and important anchors (*correction*, *interference*, etc.) are lost. As a result, VSM overemphasizes remaining generic terms (e.g., *duct*, *body*) and retrieves unrelated documents.